



UNIVERSITY OF
TORONTO

Engineering

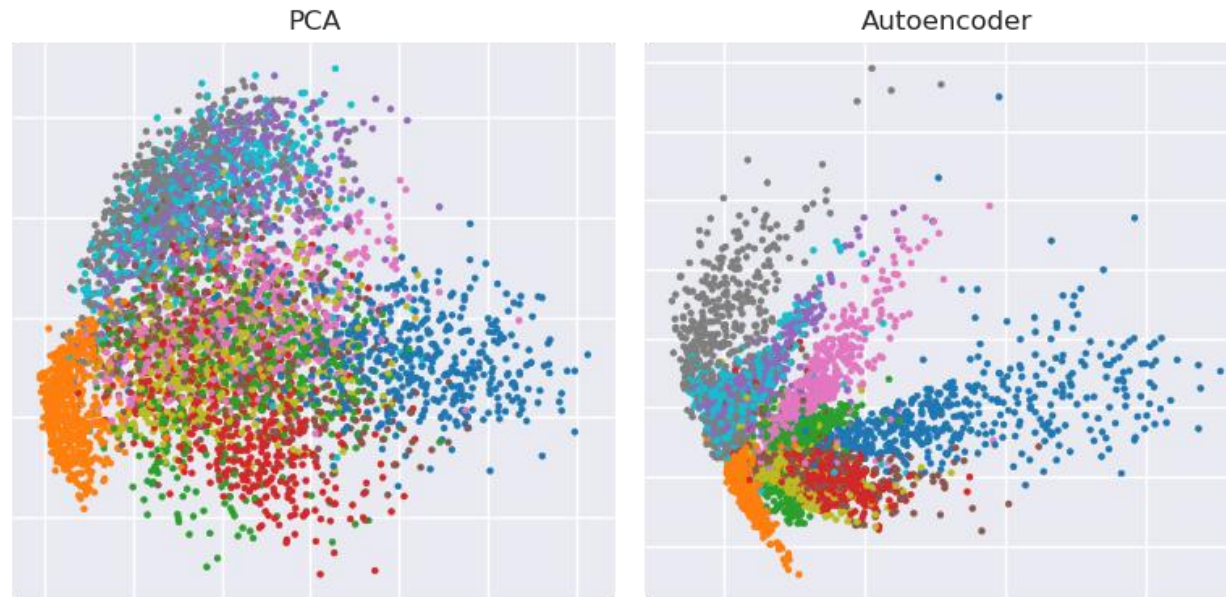
Autoencoders

Motivation

- No labels — so what do we use as the training target?
- The data itself. Reconstruct the input; learn a **representation** along the way.
- Day 3 flashback: PCA compressed data linearly.
- An autoencoder is the nonlinear version, learned by a neural network.

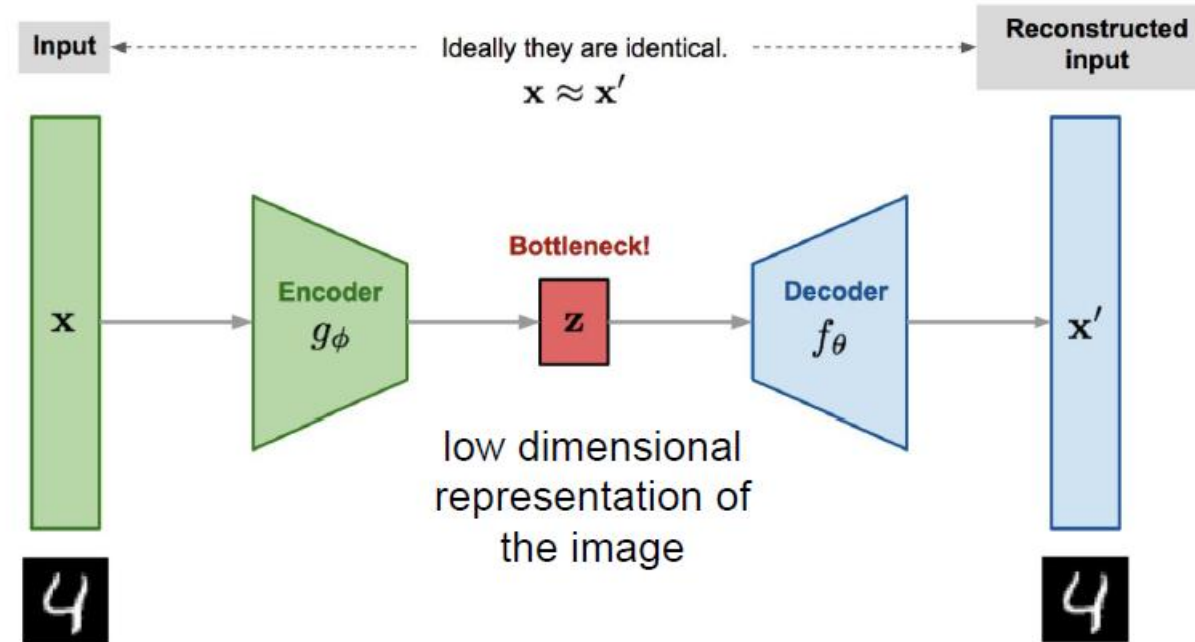
PCA vs AE

- Linear encoder/decoder + MSE $\rightarrow$ finds the **same subspace as PCA**
 $\rightarrow$ Actually...PCA is a linear AE
- Nonlinear activations $\rightarrow$ can flatten **curved** data structure PCA can't



Components

- **Encoder:** Performs dimensionality reduction and creates a latent space.
- **Decoder:** Reconstructs the input using the latent space.
- The **bottleneck** forces the network to keep only what matters
- Nothing new — just an MLP + backprop.

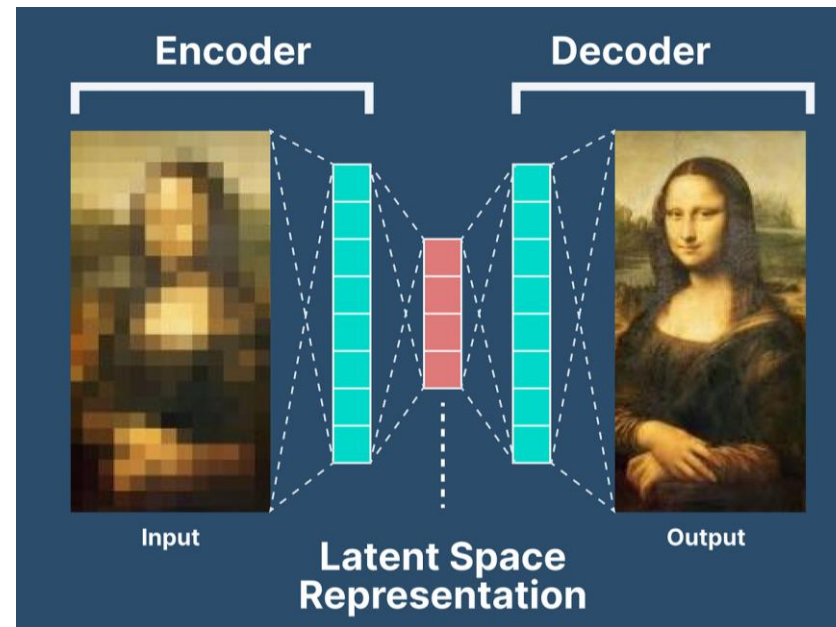


Applications

- ❖ Feature Extraction / Dimensionality Reduction
- ❖ Image Denoising
- ❖ Anomaly Detection

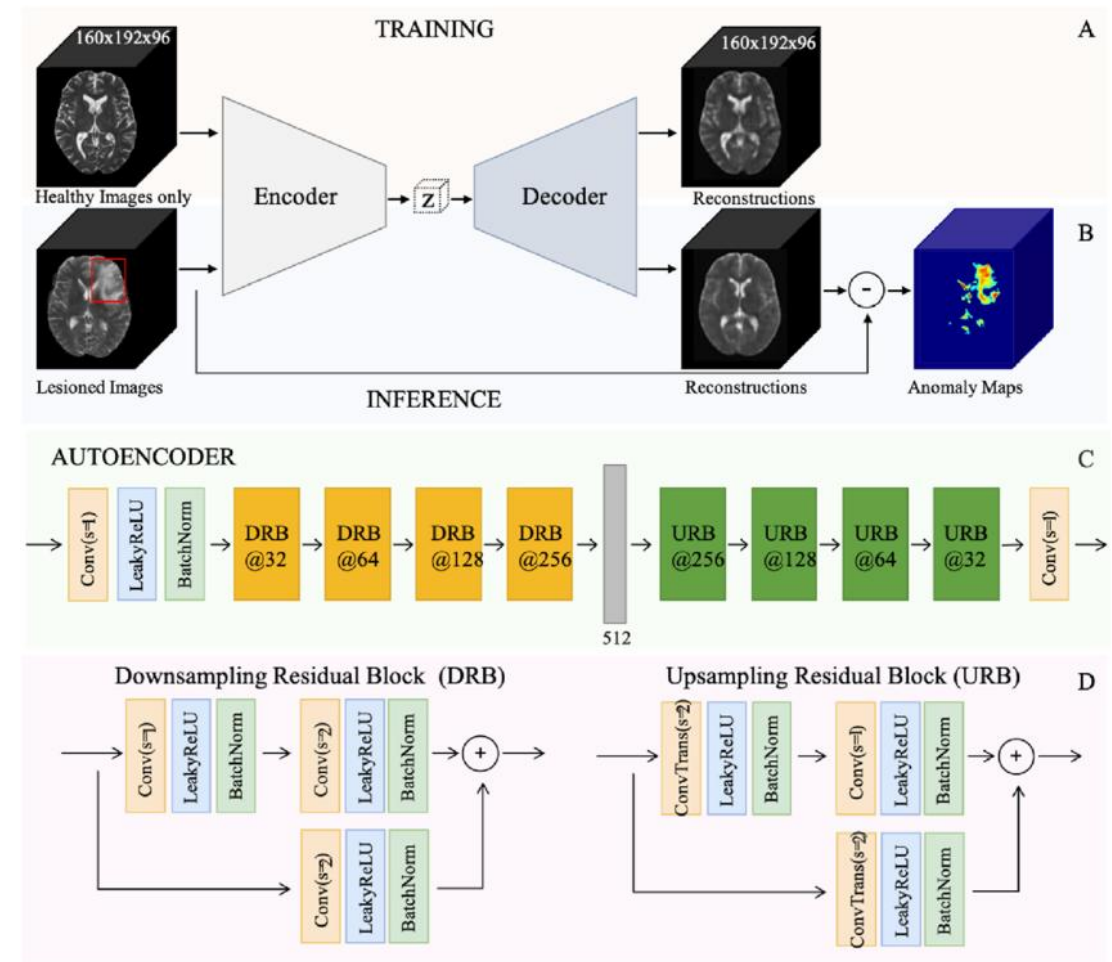
Application – Denoising

- Train: corrupted input → clean target
- Noise is unpredictable — the network's best strategy is to drop it and output the underlying structure
- Only the underlying signal survives reconstruction



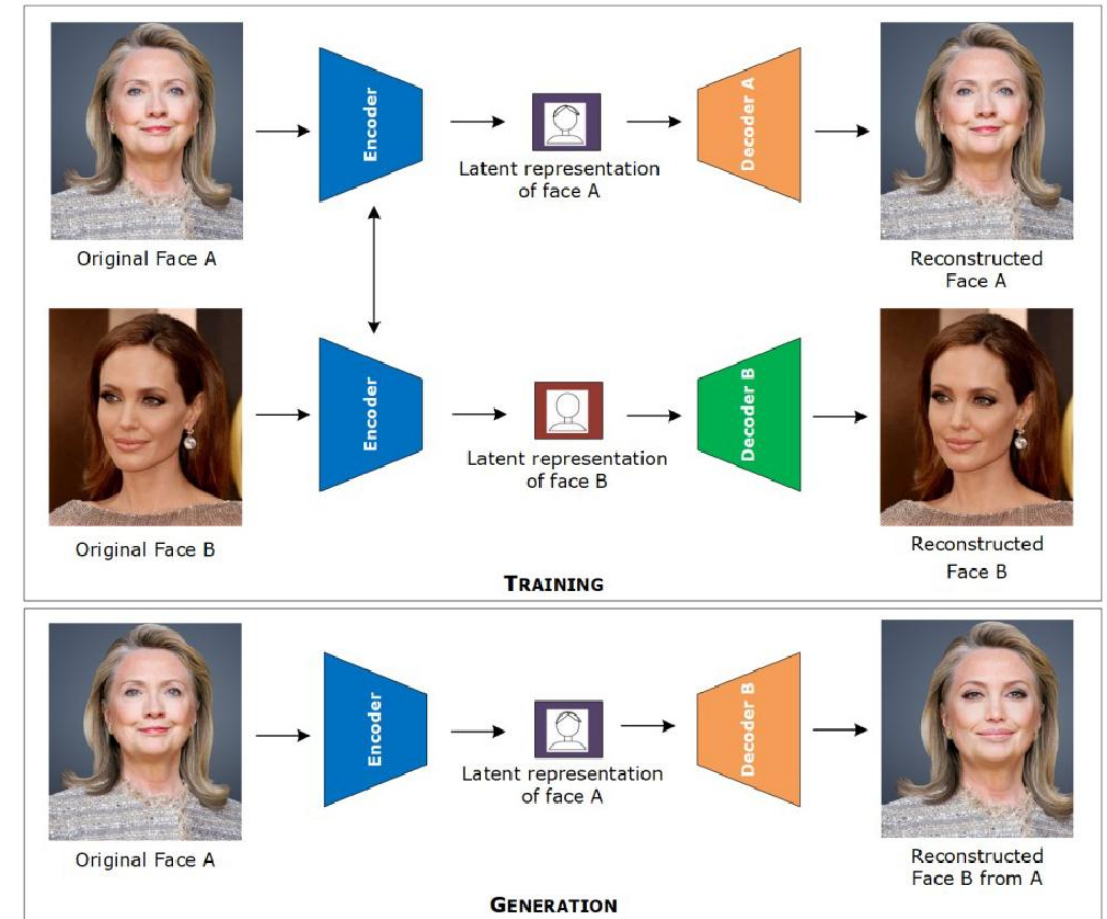
Application – Anomaly Detection

- Train on normal data only
- Anomalies reconstruct poorly → high reconstruction error
- Flag anything with error above a threshold
- Used in: manufacturing defect detection, ECG monitoring, fraud



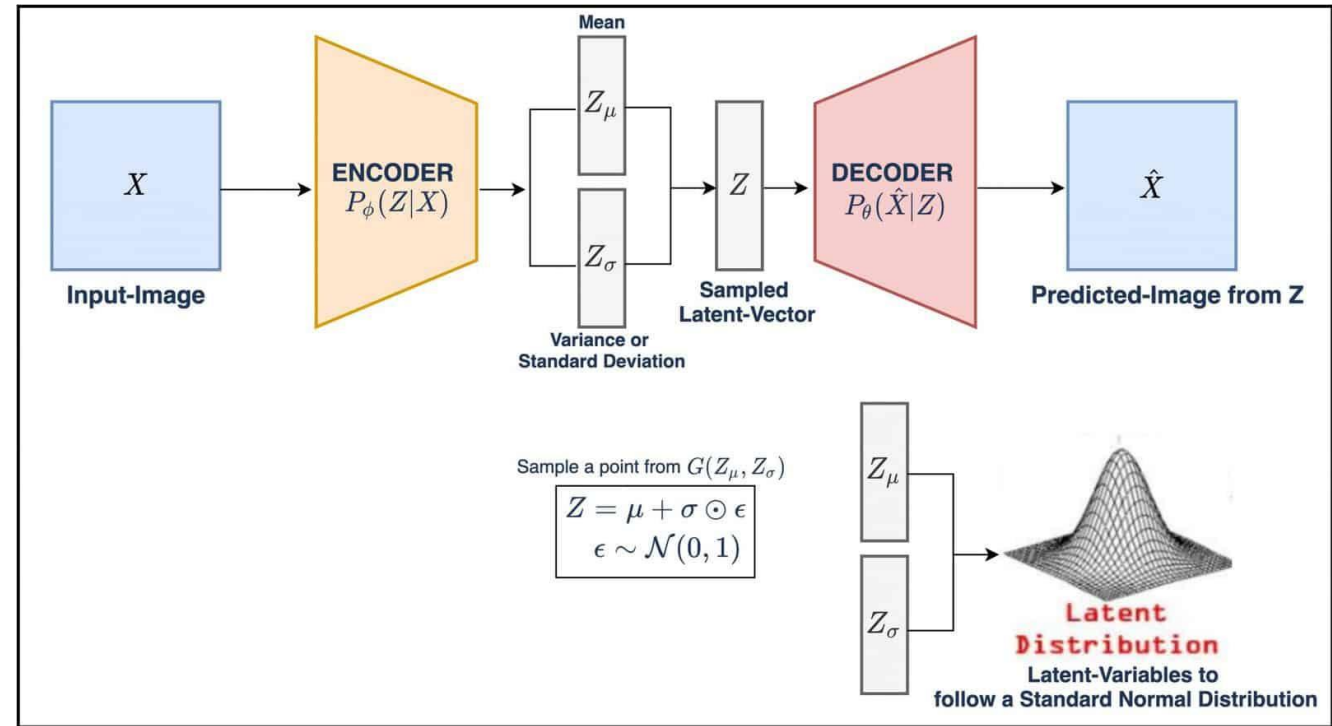
Original Deepfake

- One **shared encoder** + one **decoder per person**
- The shared latent keeps what's common: expression, pose, lighting
- **Identity lives in the decoder weights**
- At inference, swap the decoder → face A rendered as person B



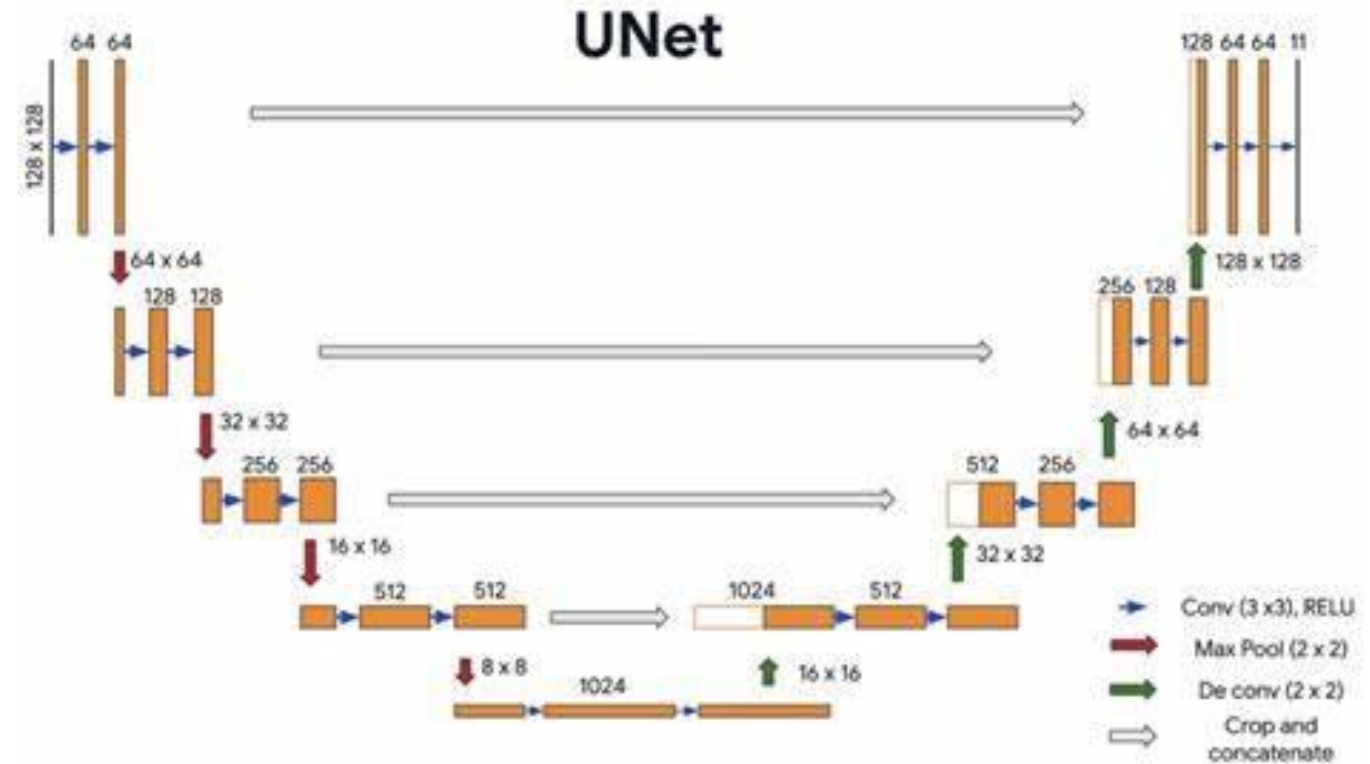
Architectures: Variational Autoencoders

- Plain AE: each input $\rightarrow$ a single point; gaps between points decode to garbage
- VAE: encode to a distribution $\rightarrow$ the latent space becomes smooth
- Smooth latent space $\rightarrow$ sample from it $\rightarrow$ generate new data
- Stable Diffusion runs on top of a VAE latent space



U-Net: A Relative, Not an Autoencoder

- Same encoder–decoder *shape* — but **not** an autoencoder
- Skip connections bypass the bottleneck
- Trained **supervised**: input $\rightarrow$ label (e.g., segmentation mask)
- Applications: medical image segmentation, diffusion model backbones





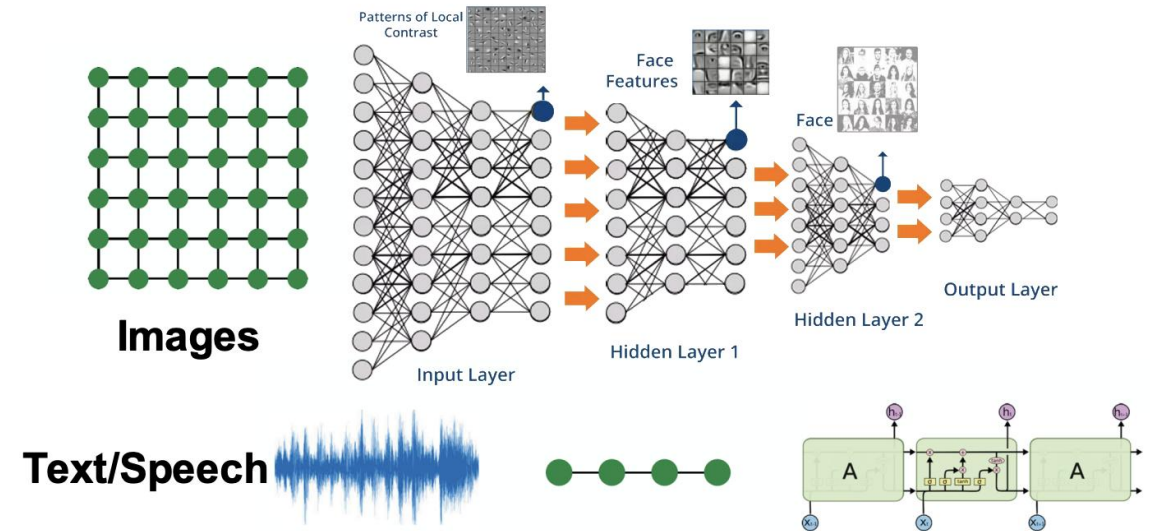
UNIVERSITY OF
TORONTO

Engineering

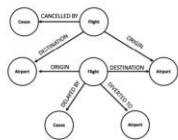
Graph Neural Networks

From Grids and Sequences to Graphs

- **Classical deep learning assumes regular structure.**
 - CNNs work on grids of pixels; RNNs on ordered tokens.
- **Most real data is irregular.**
 - Molecules, networks and maps are entities joined by relations.
 - There is no natural ordering, and degrees vary.
- **Graphs are the natural representation.**
 - GNNs extend convolution to arbitrary connectivity.



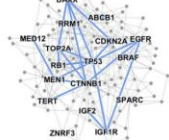
Graphs Are Everywhere



Event Graphs



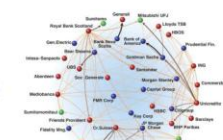
Computer Networks



Disease Pathways



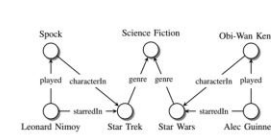
Social Networks



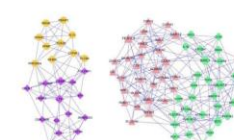
Economic Networks



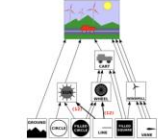
Communication Networks



Knowledge Graphs



Regulatory Networks



Scene Graphs

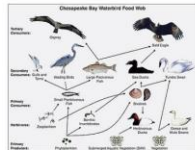


Image credit: Wikipedia

Food Webs



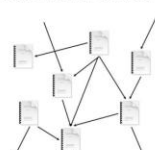
Image credit: Pinterest

Particle Networks



Image credit: visitlondon.com

Underground Networks



Citation Networks



Image credit: Missoula Current News

Internet

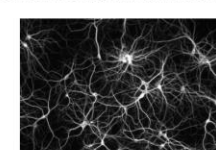


Image credit: The Conversation

Networks of Neurons

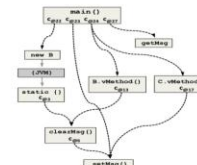


Image credit: ResearchGate

Code Graphs

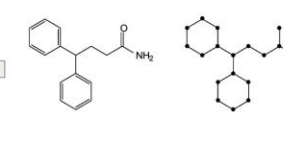


Image credit: MDPI

Molecules

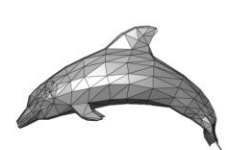


Image credit: Wikipedia

3D Shapes

Social and citation networks

Molecules and materials

Maps, meshes and 3D shapes

A Rapidly Growing Field

- GNNs went from niche to mainstream over the last decade.
- “Graph neural network” is now among the most common keywords at top ML venues.
- They are applied across chemistry, biology, recommendation, traffic and physics.

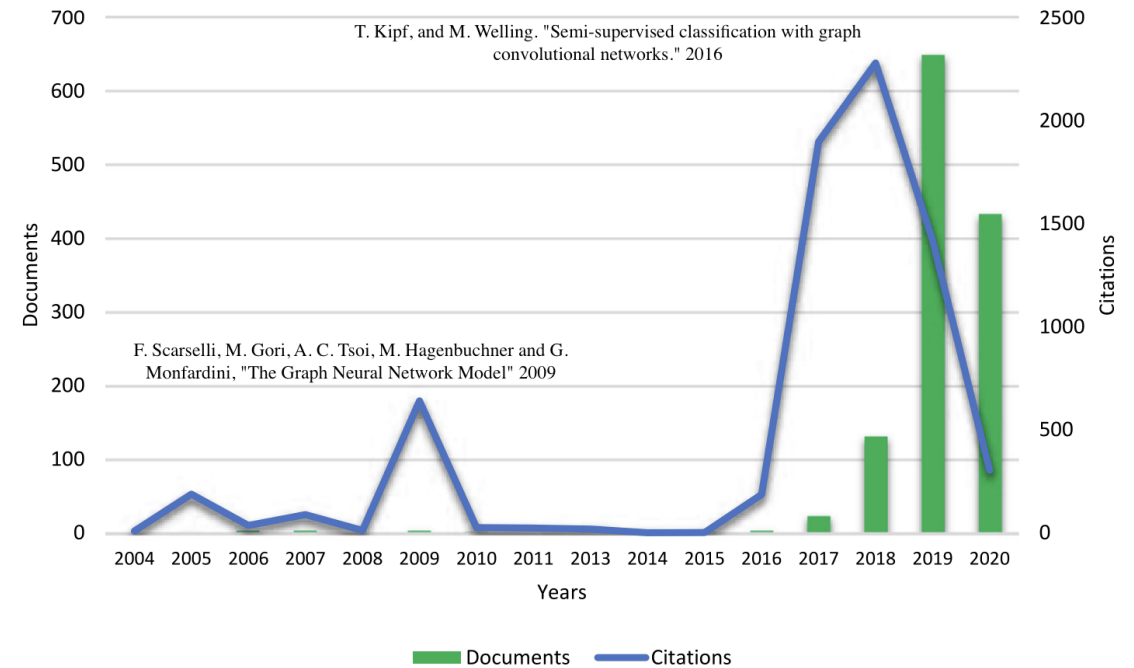
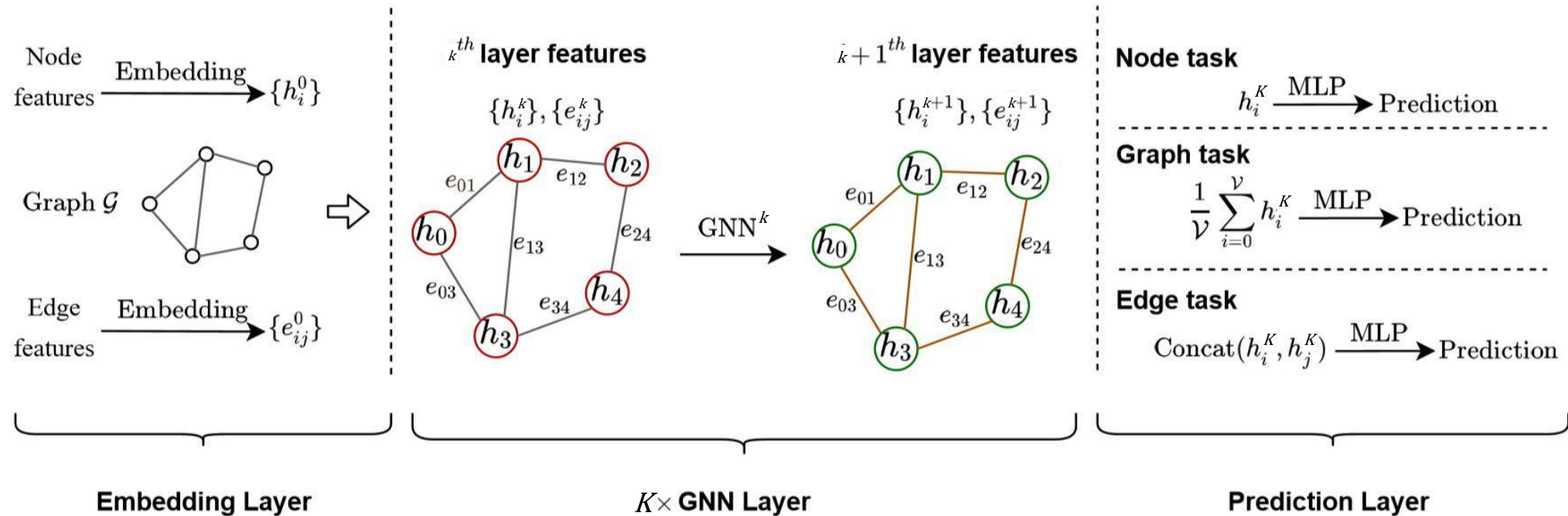


Fig. 1. Yearly number of documents and citations.

The GNN Pipeline

Input graph $\rightarrow$ message-passing layers $\rightarrow$ node embeddings $\rightarrow$ task-specific head



Node and edge level $\rightarrow$ Forecasting, routing, cybersecurity.

Link prediction $\rightarrow$ Recommendation, knowledge-graph completion.

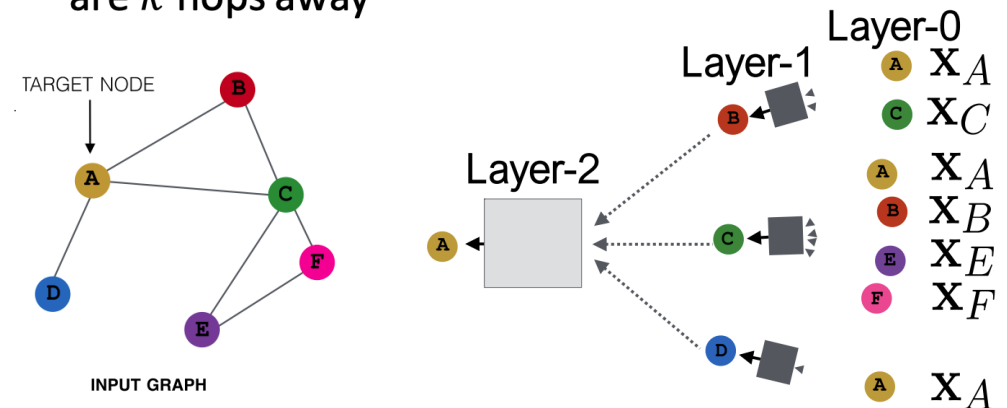
Graph level $\rightarrow$ Molecular properties, medical diagnosis.

Generation and counting $\rightarrow$ Molecule design, motif counting.

Message Passing

- **Each layer repeats two steps.**
 - Aggregate the neighbours' current embeddings.
 - Update the node's embedding with that aggregate.
- **Stacking layers grows the reach.**
 - After k layers a node has seen its k -hop neighbourhood.
- The result does not depend on node ordering.

- Model can be **of arbitrary depth**:
 - Nodes have embeddings at each layer
 - Layer-0 embedding of node v is its input feature, x_v
 - Layer- k embedding gets information from nodes that are k hops away

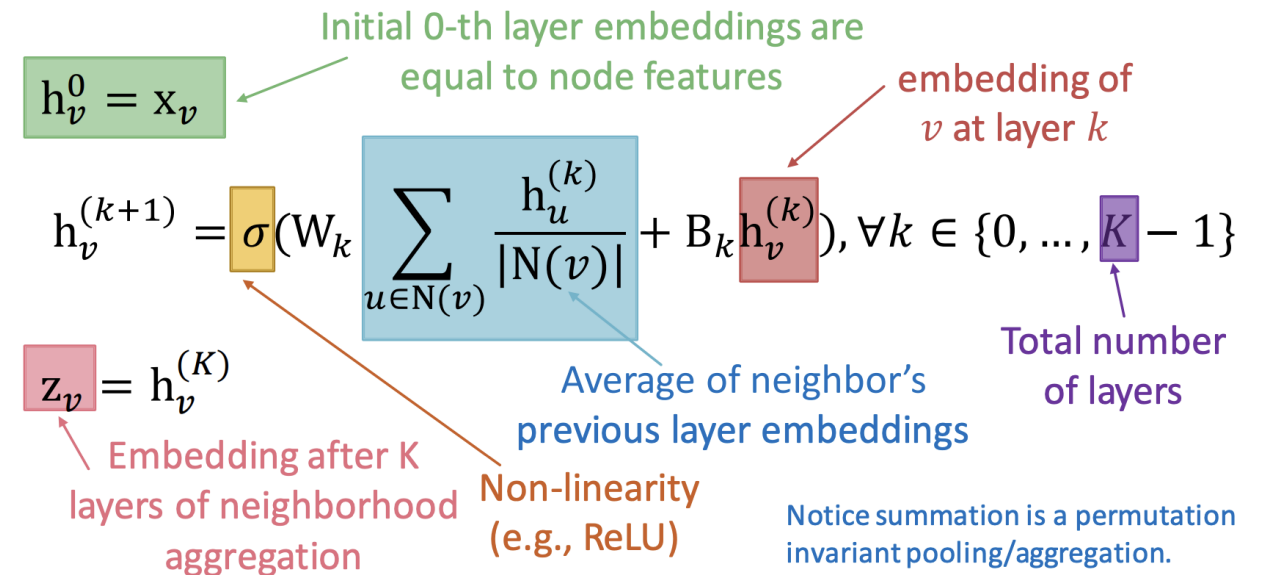


This is convolution, generalized — same weights, whoever your neighbours are.

The Graph Convolutional Network Update Rule (Canonical Modern GNN)

- Average the neighbours, then apply a learned weight and a nonlinearity.
- Symmetric normalisation keeps scales stable across node degrees.
- Weights are shared across the whole graph.
- Various aggregation choices → GAT(attention) / GraphSAGE(sampling) / GIN(sum)

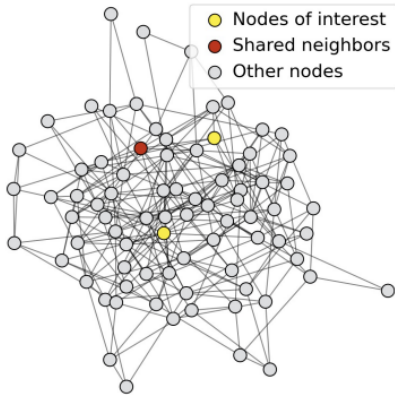
- **Basic approach:** Average neighbor messages and apply a neural network



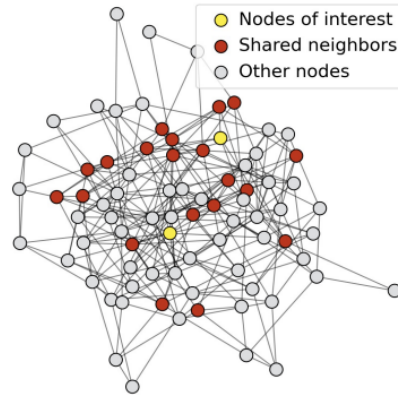
More Layers $\neq$ Better

- **Receptive field overlap for two nodes**
 - **The shared neighbors quickly grows** when we increase the number of hops (num of GNN layers)

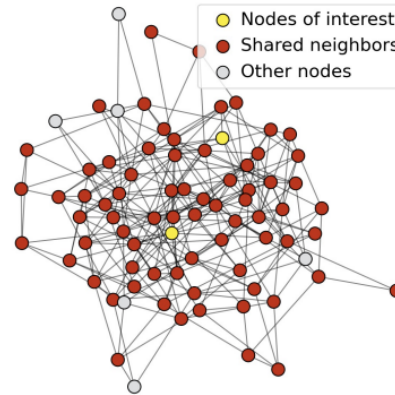
1-hop neighbor overlap
Only 1 node



2-hop neighbor overlap
About 20 nodes



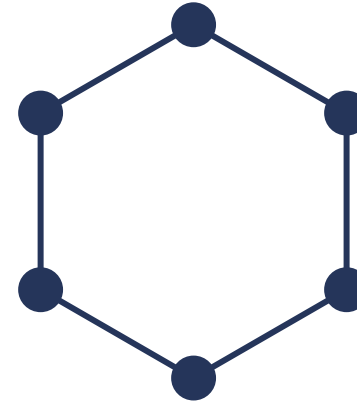
3-hop neighbor overlap
Almost all the nodes!



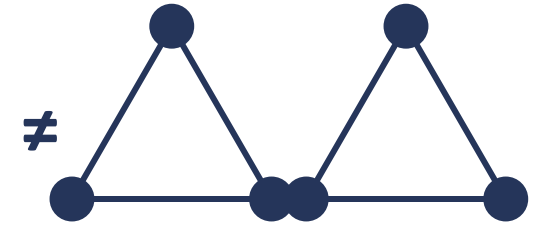
- **k layers = a k-hop field of view**
 - A few hops span most of a small-world graph.
- **Stack deeper and embeddings blur: oversmoothing.**
 - Fixes: residual connections, normalisation, jumping-knowledge.

Can a GNN Tell Two Graphs Apart?

- **A GNN describes each node by its neighbours,**
 - then its neighbours' neighbours, and so on.
- **If two graphs give the same descriptions,**
 - the GNN treats them as the same graph.
- So some genuinely different graphs are invisible to it.



One ring of six



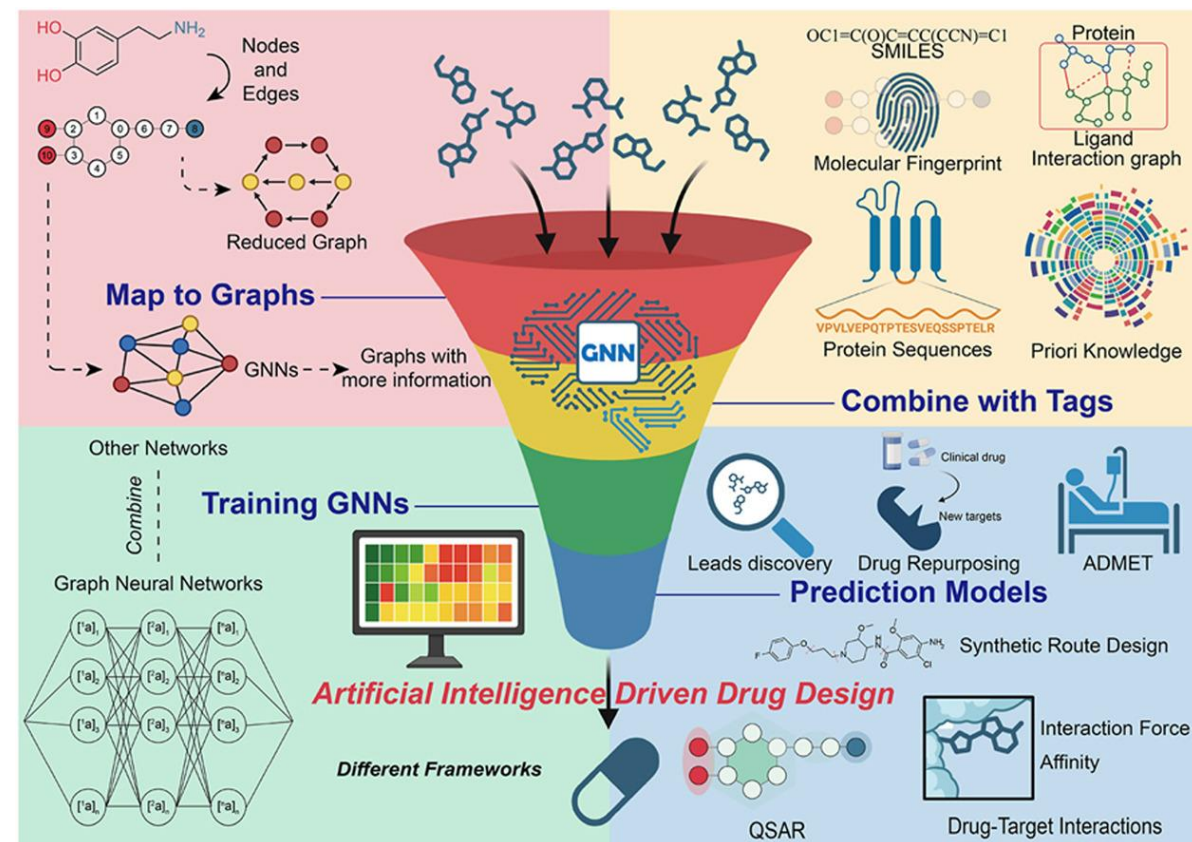
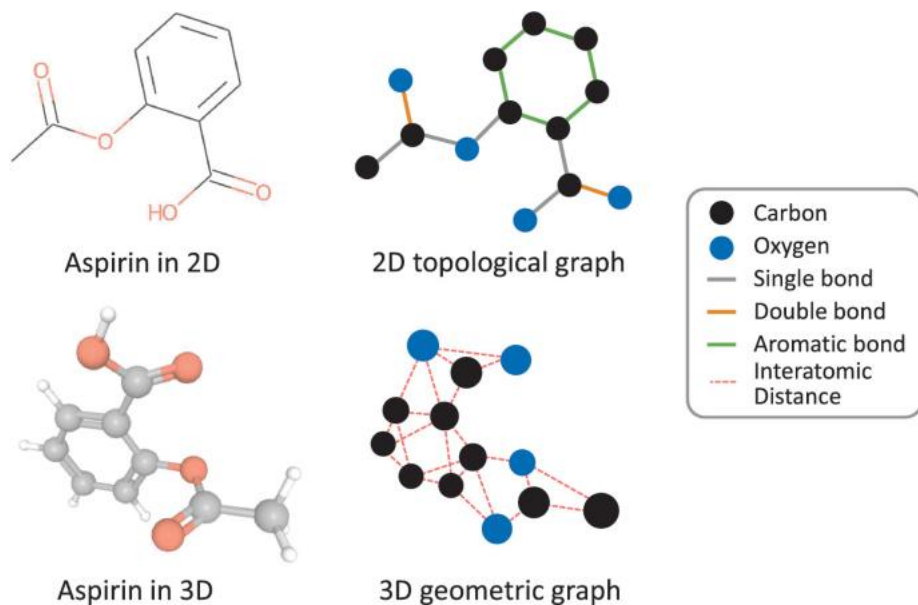
Two separate triangles

Every node has exactly two neighbours, so a standard GNN cannot tell these two apart.

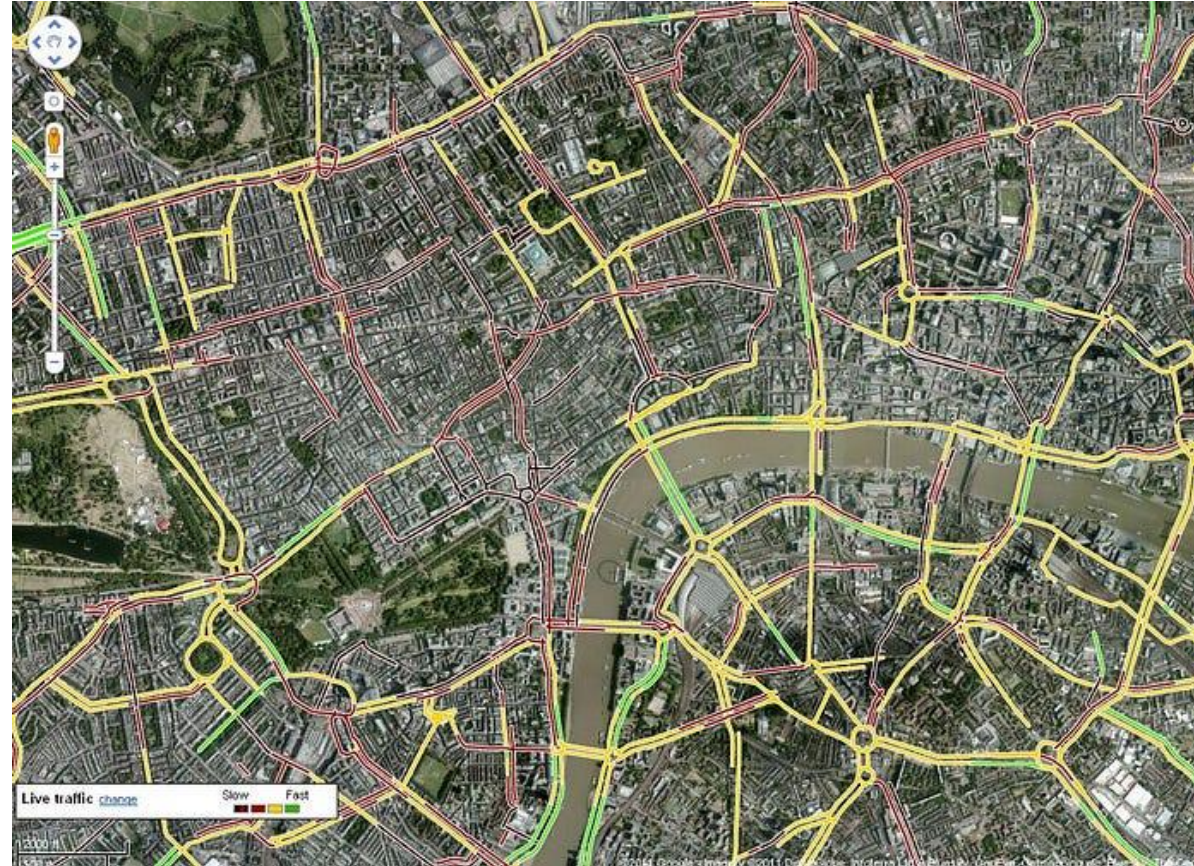
Where GNNs Struggle

- **Heterophily**
 - Message passing assumes similar neighbours, and fails when they differ.
- **Oversmoothing**
 - Deep networks blur node embeddings together.
- **Oversquashing**
 - Long-range signal is crushed through bottlenecks.
- **Limited expressiveness**
 - Some genuinely different graphs look identical to a GNN.

Application – Molecules & Drug Discovery



Application – Google Maps ETA





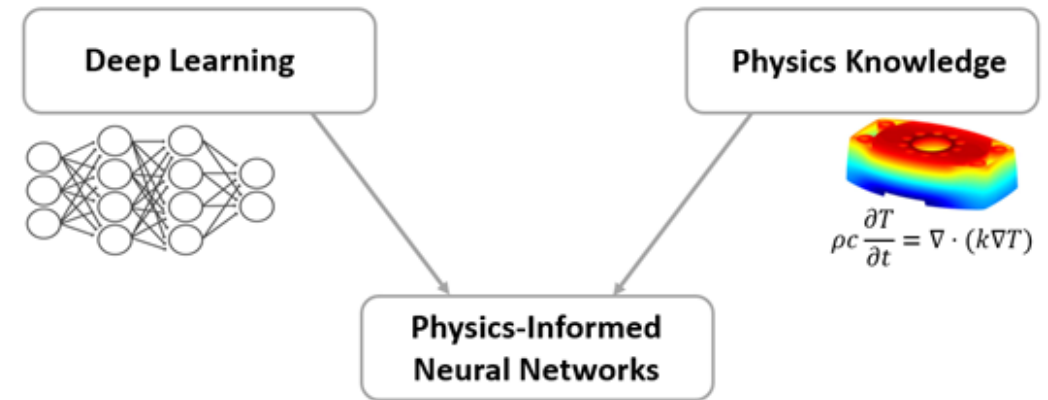
UNIVERSITY OF
TORONTO

Engineering

Physics-Informed Neural Networks

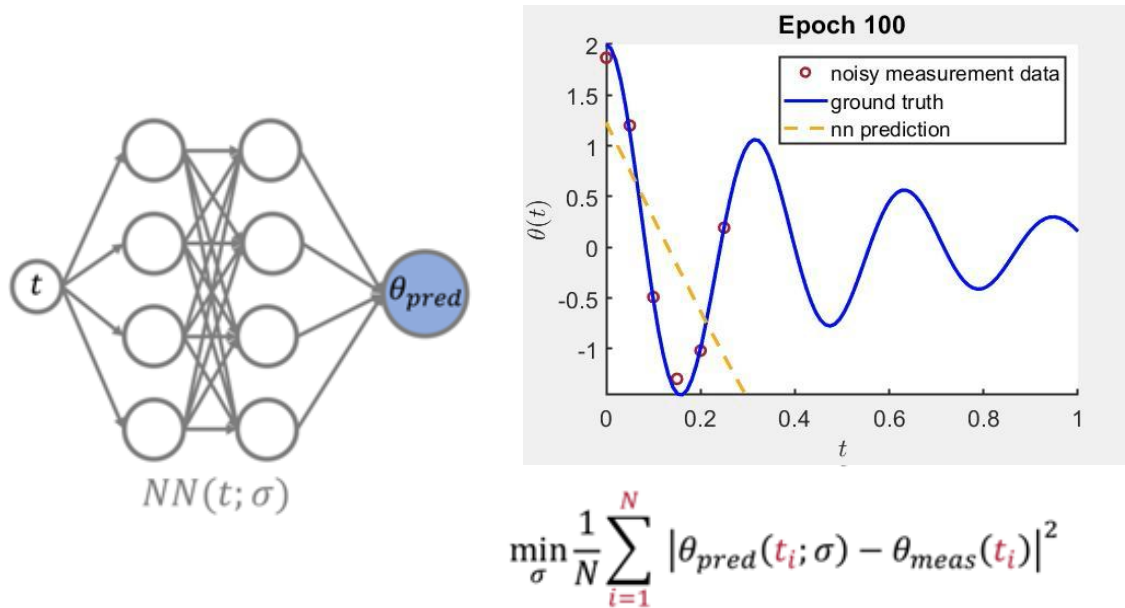
Why Physics-Informed Learning?

- **Standard networks fit data and ignore the physics behind it.**
- **Yet many systems obey known ODEs and PDEs.**
 - Fluids, heat, waves and structures all follow physical law.
- **PINNs add those laws to the loss.**
 - Better generalisation, less data, physically consistent (approximately) output.



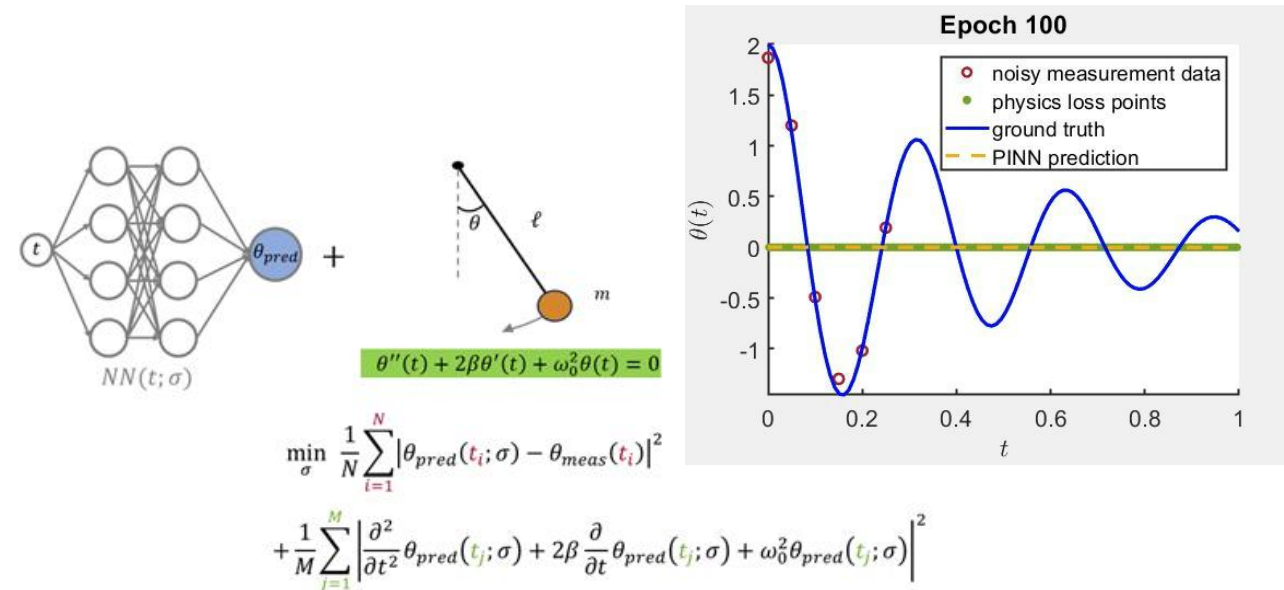
Intuition: The Pendulum

Plain neural network



Fits the samples, but drifts from the true motion elsewhere.

Physics-informed network



The equation of motion is a loss term, so the fit stays physical even with sparse data.

From a Niche Idea to Scientific ML

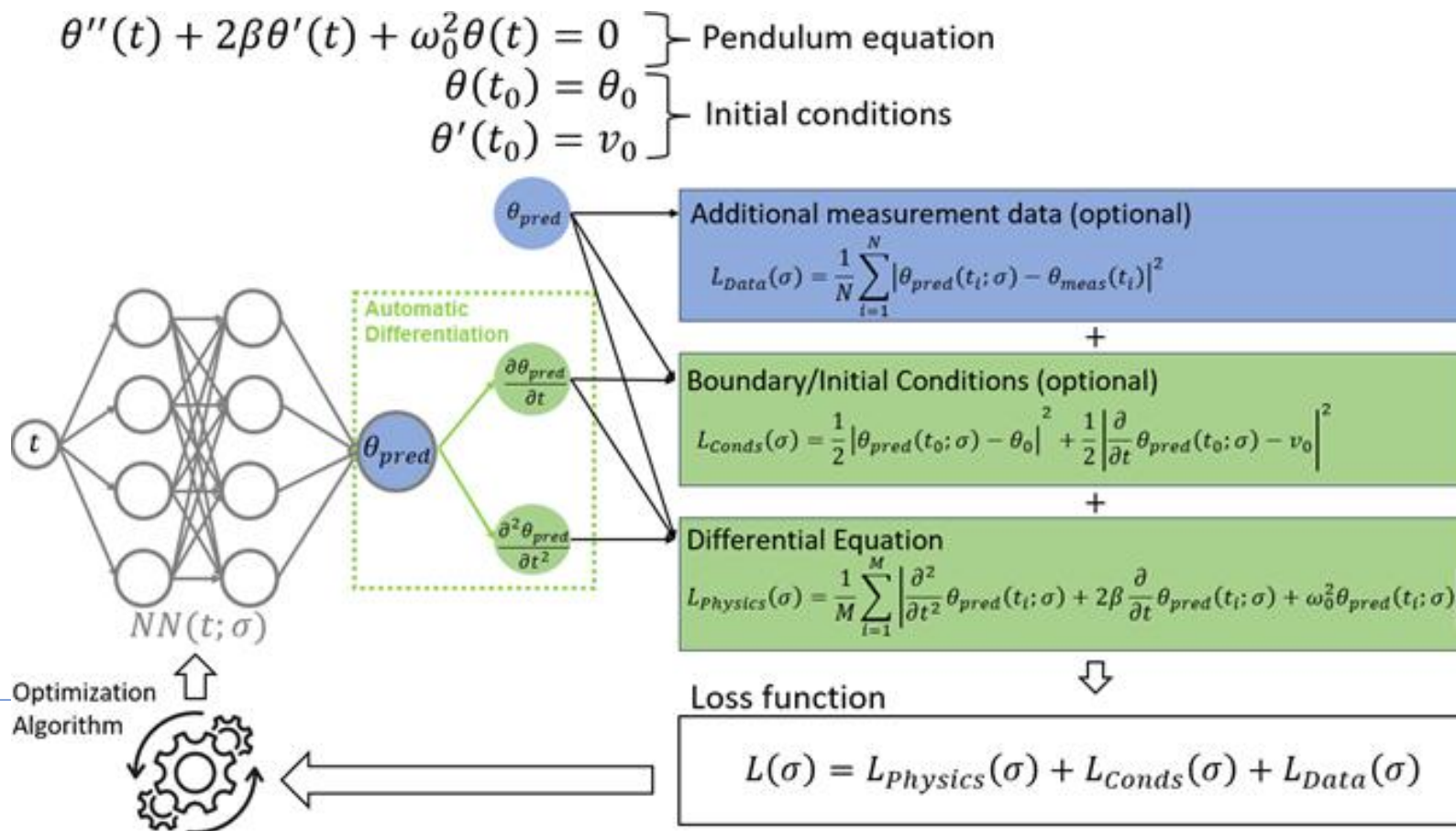
- **1998 — First steps**
 - Lagaris et al. solve differential equations with neural nets, limited by the compute of the day.
- **2017–19 — Modern revival**
 - Raissi, Perdikaris and Karniadakis reframe and popularise PINNs.
- **Today — Scientific ML**
 - A standard tool for simulating and inferring physical systems.
- **What made PINNs possible:** automatic differentiation, powerful GPUs, and mature optimizers such as Adam.

Where PINNs Shine

- **Expensive simulations**
 - Replace costly FEM or CFD solvers with fast approximations (surrogate modelling).
- **Sparse or noisy data**
 - Use physics to fill gaps, as in flow reconstruction or seismology.
- **Inverse problems**
 - Infer parameters that cannot be measured directly.

Ingredients

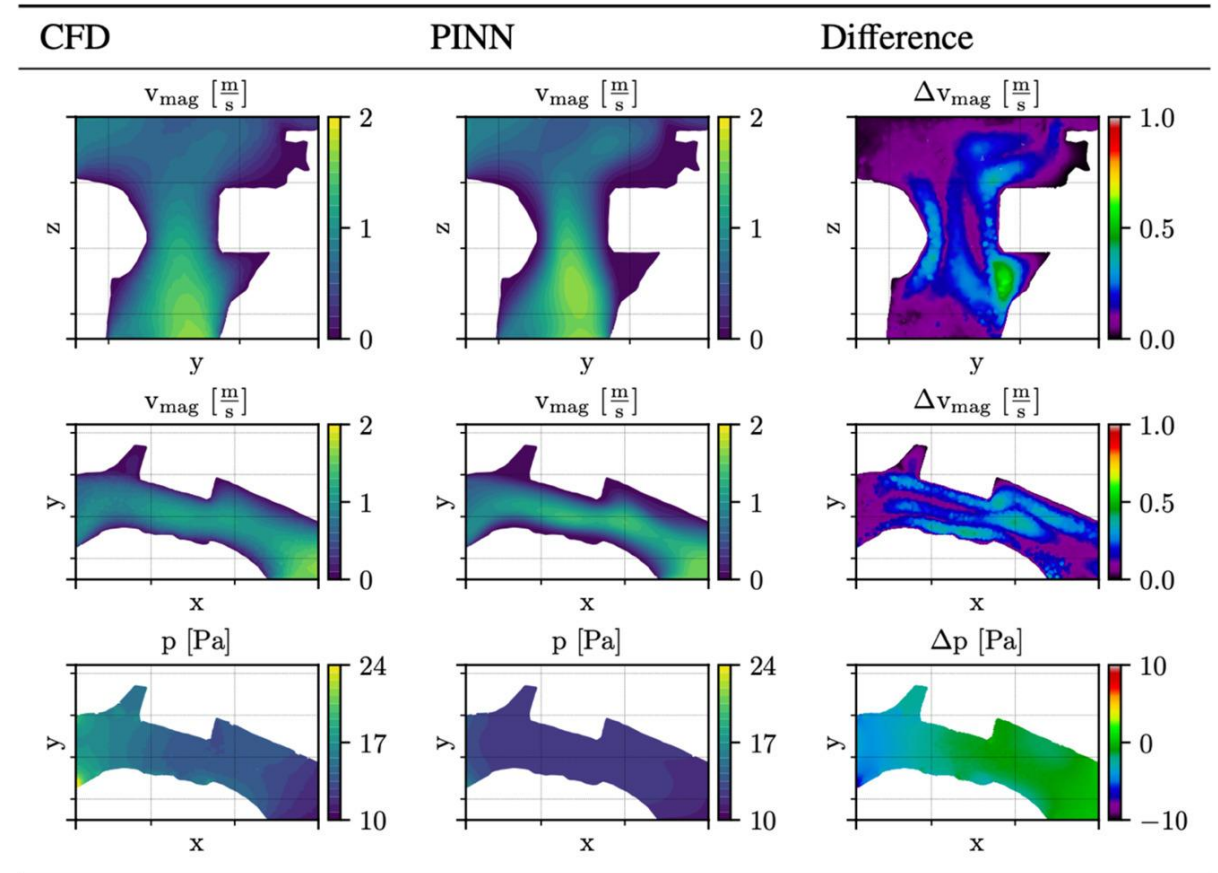
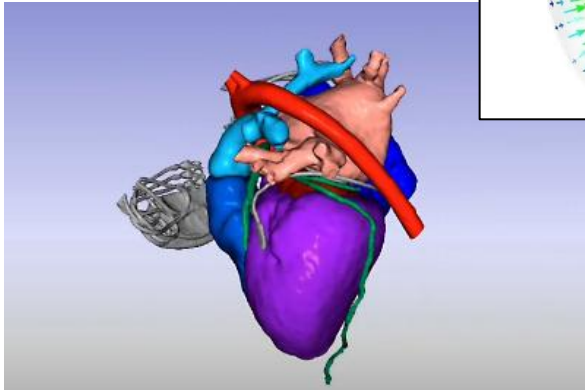
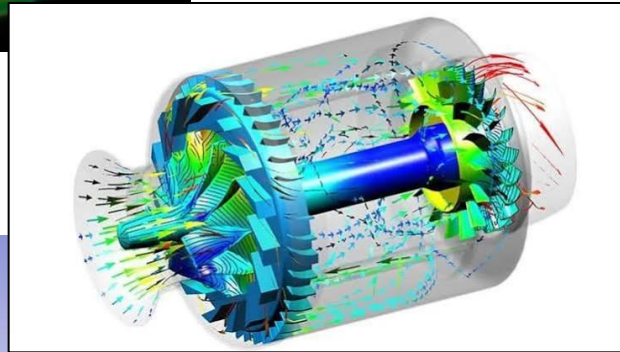
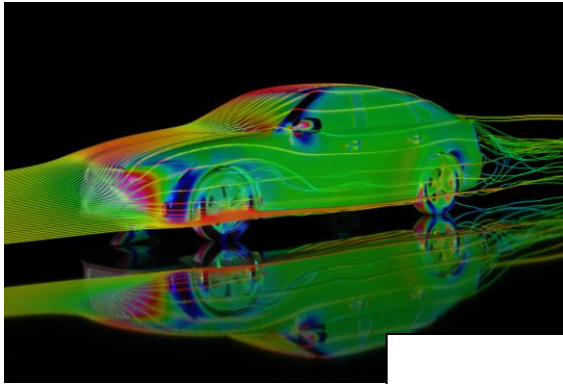
- Derivatives come straight from the computational graph (Autodiff).
- The objective: $L_{\text{total}} = L_{\text{data}} + L_{\text{IC/BC}} + L_{\text{phys}}$



Forward and Inverse Modeling

- **Forward:** given parameters, compute the solution field.
 - e.g. given conductivity K , solve for the temperature.
 - Replaces an expensive simulator.
- **Inverse:** given sparse observations, infer hidden parameters.
 - e.g. estimate K from measured temperatures.
 - Ill-posed: sensitive to noise and often non-unique.
- Trainable quantities can include not just the weights, but unknown physical parameters.

CFD Surrogate Modelling



The Hard Parts

- **Training difficulty**
 - Stiff, multi-scale dynamics and heavy problem-specific tuning.
- **Computational cost**
 - Residuals and high-order derivatives are expensive to evaluate.
- **Generalisation**
 - A trained PINN usually fits one setup and must be retrained.
- **Inverse modeling**
 - Ill-posed and non-unique; several parameters need more data.

The Same Idea in Other Domains

- **Physics**
 - Conservation laws and PDEs must hold.
- **Finance**
 - Prices must be arbitrage-free.
- **Combinatorial optimization**
 - Solutions must stay feasible.
- **When to reach for a PINN:** the rules matter more than raw accuracy, and labelled data is scarce.



UNIVERSITY OF
TORONTO

Engineering

Thank you.